

计算机系统实验报告

实验二：系统移植实验

姓 名： 韦畅

学 号： 2352628

指导教师： 郭玉臣

实验日期： 2026 年 5 月 27 日

目录

1	实验目的	3
2	实验环境	3
3	系统架构	4
3.1	整体架构	4
3.2	地址映射	4
3.3	时钟与复位	5
3.4	Wishbone 总线互连	5
4	核心模块设计	6
4.1	GPIO 模块	6
4.2	UART 模块	7
4.3	SPI Flash 控制器	7
4.3.1	SPI 协议与时钟输出	7
4.3.2	读取命令与时序	8
4.3.3	控制器状态机	8
4.4	DDR2 SDRAM 控制器	9
4.4.1	MIG IP 核配置	9
4.4.2	用户接口信号	9
4.4.3	DDR2 Wishbone 桥接	10
5	软件移植	11
5.1	BootLoader 修改	11
5.2	μ C/OS-II 系统源码修改	11
5.3	交叉编译与 OS.bin 生成	11
6	下板验证	12
6.1	烧录流程	12

6.2	验证结果	13
6.2.1	串口输出	13
6.2.2	数码管显示	13
6.2.3	LED 状态指示	14
7	遇到的问题与解决方法	15
7.1	时序违例	15
7.2	SPI Flash 控制器未自动启动	15
7.3	PLL 锁定信号未接入复位	16
7.4	DDR2 初始化轮询死锁	16
7.5	SPI Flash 采样相位错位（核心问题）	16
8	实验总结	18
8.1	实验收获	18
8.2	实验结果	19

1 实验目的

1. 在实验一已完成的支持 89 条指令的 OpenMIPS CPU 基础上，搭建包含 Wishbone 总线互连、GPIO、UART、SPI Flash 和 DDR2 SDRAM 的完整 SoC 系统。
2. 理解 Wishbone 片上总线协议，掌握共享总线互连方式下主从设备的地址解码与仲裁机制。
3. 了解 SPI Flash 的读取时序，实现通过 STARTUPE2 原语输出 SPI 时钟、从 Flash 中一次性读取 4 字节数据的控制器。
4. 了解 DDR2 SDRAM 的基本工作原理，使用 Xilinx MIG (Memory Interface Generator) IP 核实现 DDR2 接口，完成 DDR2 的初始化校准与读写操作。
5. 掌握 BootLoader 的工作流程：从 SPI Flash 中读取 BootLoader 和操作系统映像到 DDR2 SDRAM，然后跳转到 SDRAM 执行。
6. 在 Ubuntu 环境下搭建 MIPS 交叉编译工具链，修改 BootLoader 和 $\mu\text{C}/\text{OS-II}$ 源码中的 UART 分频参数，完成操作系统的交叉编译与二进制文件生成。
7. 将 BootLoader+ $\mu\text{C}/\text{OS-II}$ 合并的二进制文件烧录到 SPI Flash 中，下载比特流到 FPGA 开发板，通过串口终端 (SuperCom) 观察到 $\mu\text{C}/\text{OS-II}$ 的启动输出。

2 实验环境

表 1: 实验环境配置

项目	配置
操作系统 (Vivado)	Windows 11
操作系统 (交叉编译)	Ubuntu 12.04 (虚拟机)
开发工具	Xilinx Vivado 2024.2
串口工具	SuperCom
FPGA 开发板	Digilent NEXYS 4 DDR
FPGA 芯片	Xilinx Artix-7 xc7a100tcs9324-1
DDR2 芯片	MT47H64M16HR-25:H
SPI Flash	Spansion S25FL128S
硬件描述语言	Verilog HDL
交叉编译工具链	mips-sde-elf (GCC 4.3)

3 系统架构

3.1 整体架构

本实验在实验一的 OpenMIPS CPU 基础上，构建了一个完整的片上系统（SoC），系统结构如图 1 所示。系统采用 Wishbone 共享总线互连方式，CPU 作为主设备通过 Wishbone 总线连接四个从设备：DDR2 SDRAM 控制器、UART 控制器、GPIO 模块和 SPI Flash 控制器。

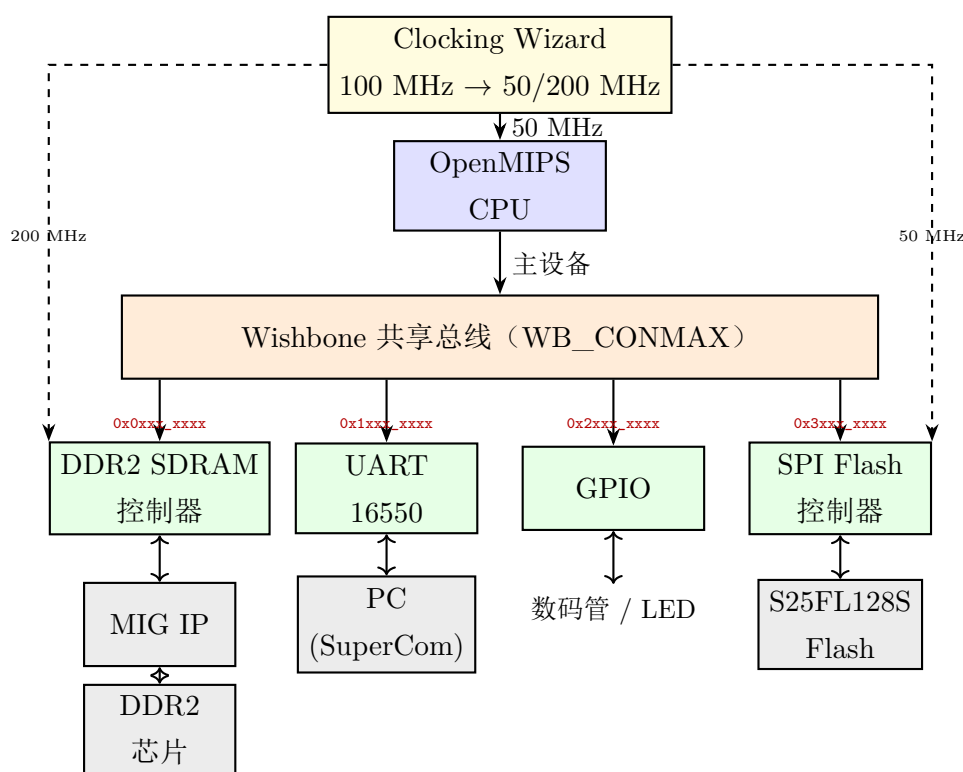


图 1: 系统整体架构

3.2 地址映射

系统使用地址高 4 位（`wb_addr_i[31:28]`）进行从设备选择，地址映射关系如表 2 所示。

表 2: 地址映射表

地址范围	高 4 位	从设备	说明
0x00000000--0x0FFFFFFF	4'b0000	DDR2 SDRAM	256 MB 主存储器
0x10000000--0x1FFFFFFF	4'b0001	UART16550	串口通信
0x20000000--0x2FFFFFFF	4'b0010	GPIO	通用输入输出
0x30000000--0x3FFFFFFF	4'b0011	SPI Flash	程序存储

CPU 复位后从地址 0x30000000 (SPI Flash 起始地址) 开始取指执行 BootLoader。BootLoader 首先将存储在 Flash 中的 μ C/OS-II 操作系统映像搬移到 DDR2 SDRAM 中, 然后跳转到 SDRAM 执行操作系统。

3.3 时钟与复位

系统时钟由板载 100MHz 晶振通过时钟向导 (Clocking Wizard) IP 核产生两个时钟域:

表 3: 时钟配置

时钟	频率	用途
clk_out1	50 MHz	Wishbone 总线、CPU、UART、GPIO、Flash 控制器
clk_out2	200 MHz	DDR2 MIG 参考时钟

复位逻辑采用板载复位按钮和 PLL 锁定信号的逻辑或:

```
1 assign rst = ~rst_n | ~locked;
```

Listing 1: 复位逻辑

其中 `rst_n` 为板载复位按钮 (低电平有效), `locked` 为 Clocking Wizard 输出的 PLL 锁定信号。该设计确保在 PLL 未完成锁定时系统保持复位状态, 避免在时钟不稳定时出现未知行为。

3.4 Wishbone 总线互连

本实验采用 Wishbone B4 规范中的共享总线 (Shared Bus) 互连方式, 由 OpenCores 提供的 WB_CONMAX 模块实现。在该方式下, CPU 作为唯一的主设备 (Master), 通过地址高 4 位选择不同的从设备 (Slave), 同一时刻只有一个从设备与主设备通信。

Wishbone 总线信号包括：

表 4: Wishbone 总线关键信号

信号	方向	说明
CLK_I	输入	总线时钟
RST_I	输入	总线复位
DAT_O/DAT_I	双向	数据总线（32 位）
ADR_O	输出	地址总线（32 位）
WE_O	输出	写使能（1= 写，0= 读）
SEL_O	输出	字节选择（4 位）
CYC_O	输出	总线周期有效
STB_O	输出	选通有效
ACK_I	输入	从设备应答

一次典型的读总线事务流程为：主设备在第一个时钟上升沿输出地址和读控制信号（CYC_O、STB_O 同时置高），从设备在下一个时钟上升沿将数据放到 DAT_I 并拉高 ACK_I，主设备采样数据后释放总线周期信号。写事务的流程类似，区别在于主设备同时输出写数据。

4 核心模块设计

4.1 GPIO 模块

GPIO（General Purpose Input/Output）模块提供通用的输入输出接口。本实验使用 OpenCores 提供的 GPIO IP 核，连接到 Wishbone 从设备接口 `slv_sel[2]`。CPU 通过 GPIO 读取外部输入信号或控制外部输出设备。

在本实验中，GPIO 的主要用途是：

- **输入：**读取 SDRAM 初始化完成信号（`sdram_init_done`），BootLoader 通过轮询 GPIO 的第 16 位等待 DDR2 完成校准。
- **输出：**驱动数码管显示， μ C/OS-II 启动后通过 GPIO 输出数据到数码管进行递增显示。

4.2 UART 模块

UART (Universal Asynchronous Receiver/Transmitter) 模块实现异步串行通信，用于将 $\mu\text{C}/\text{OS-II}$ 的输出信息发送到 PC 端的串口终端。本实验使用 OpenCores 提供的 UART16550 IP 核，连接到 Wishbone 从设备接口 `slv_sel[1]`。

UART 控制器内部分频系数设置为 651，对应 4800 波特率 ($50\text{ MHz} \div 651 \div 16 \approx 4800$)。数据格式为 8 位数据位、无校验位、1 位停止位 (8-N-1)。

开发板上的 UART 引脚连接为：

表 5: UART 引脚映射

功能	FPGA 引脚	Nexys4 DDR 标识
TXD (发送)	D4	JD1
RXD (接收)	C4	JD2
CTS	D3	JD3
RTS	E5	JD4

通过 USB-UART 桥接芯片，开发板的 UART 接口映射为 PC 的 COM3 串口。

4.3 SPI Flash 控制器

SPI Flash 控制器负责从 SPI Flash 中读取 BootLoader 和操作系统映像。本实验使用的 SPI Flash 芯片为 Spansion S25FL128S，容量为 128 Mbit (16 MB)，通过 SPI (Serial Peripheral Interface) 协议进行通信。

4.3.1 SPI 协议与时钟输出

SPI 通信使用 4 根信号线：

- **sck**: SPI 时钟
- **cs_n**: 片选信号 (低电平有效)
- **si** (MOSI): 主机到从机的数据线
- **so** (MISO): 从机到主机的数据线

由于 SPI Flash 的 SCK 引脚直接连接到 FPGA 的专用配置引脚，普通 IO 无法驱动，因此使用 Xilinx Artix-7 的 STARTUPE2 原语，通过其 USRCCLK0 端口输出 SPI 时钟：

```

1 STARTUPE2 #(
2     .PROG_USR("FALSE"),
3     .SIM_CCLK_FREQ(0)
4 ) STARTUPE2_inst (
5     .CFGCLK      (),
6     .CFGMCLK     (),
7     .EOS         (),
8     .PREQ        (),
9     .CLK         (clk),
10    .GSR         (1'b0),
11    .GTS         (1'b0),
12    .KEYCLEARB   (1'b0),
13    .PACK        (1'b0),
14    .USRCCLK0    (flash_clk),
15    .USRCCLKTS   (1'b0),
16    .USRDONEO    (),
17    .USRDONETS   (1'b1)
18 );

```

Listing 2: STARTUPE2 原语连接

4.3.2 读取命令与时序

Flash 读取操作使用 READ 命令（指令码 0x03），一次读取 4 字节（32 位）数据的时序流程如下：

1. 拉低 `cs_n` 选中 Flash 芯片。
2. 通过 `si` 线依次发送 READ 指令（8 位）和 24 位读取地址，共计 32 个 SCK 时钟周期。
3. Flash 芯片通过 `so` 线依次返回 4 字节数据，每字节 8 位，共 32 个 SCK 时钟周期。
4. 拉高 `cs_n` 结束本次读取。

4.3.3 控制器状态机

Flash 控制器采用有限状态机实现，主要状态包括：

表 6: Flash 控制器状态机

状态	功能
IDLE	空闲，等待 CPU 发起读请求
START	发送 READ 指令（8 位）
ADDR1-ADDR3	发送 24 位读取地址
READ_DATA1-READ_DATA5	接收 4 字节数据并逐位移位累积
DONE	拉高 <code>cs_n</code> ，通过 Wishbone 返回数据

在 READ_DATA 阶段，控制器使用 `datain_shift` 移位寄存器逐位累积从 `so` 线接收到的数据。每 8 位构成一个完整字节，4 个字节拼接为 32 位的 `read_data`，通过 Wishbone 总线返回给 CPU。

4.4 DDR2 SDRAM 控制器

DDR2 SDRAM 控制器基于 Xilinx MIG (Memory Interface Generator) IP 核实现，为 CPU 提供 256 MB 的主存储空间。

4.4.1 MIG IP 核配置

MIG IP 核的关键配置参数：

表 7: MIG IP 核关键参数

参数	值
存储器类型	DDR2 SDRAM
数据位宽	16 位（外部总线）
突发长度	8（一次读/写 128 位数据）
用户接口数据宽度	128 位
用户接口时钟	<code>ui_clk</code> （由 MIG 内部 PLL 生成）
地址宽度	27 位

4.4.2 用户接口信号

MIG 提供简化的用户侧接口（APP 接口），主要信号包括：

表 8: MIG APP 接口信号

信号	方向	说明
app_addr[26:0]	输入	访问地址
app_cmd[2:0]	输入	命令：001= 读，000= 写
app_en	输入	命令使能
app_rdy	输出	命令就绪（可接受新命令）
app_wdf_data[127:0]	输入	写数据
app_wdf_end	输入	写数据最后一拍
app_wdf_wren	输入	写数据有效
app_wdf_rdy	输出	写数据通道就绪
app_rd_data[127:0]	输出	读数据
app_rd_data_valid	输出	读数据有效
app_sr_req	输入	自刷新请求
app_ref_req	输入	刷新请求
ui_clk	输出	用户侧时钟
ui_clk_sync_rst	输出	用户侧复位
init_calib_complete	输出	初始化校准完成

MIG 在 FPGA 配置完成后自动进行 DDR2 的初始化校准序列。校准完成后,init_calib_complete 信号拉高,表示 DDR2 已准备好接受读写操作。本实验中该信号通过 GPIO 的第 16 位传递给 CPU, BootLoader 通过轮询该位等待 DDR2 就绪。

4.4.3 DDR2 Wishbone 桥接

由于 MIG 的用户侧接口（APP 接口）与 Wishbone 总线协议不同,需要一个桥接模块完成协议转换。桥接模块的主要功能包括:

- 将 Wishbone 单次 32 位读写请求转换为 MIG 的 128 位突发读写。
- 写操作时,将 32 位数据对齐拼接到 128 位的 app_wdf_data 中。
- 读操作时,从 128 位的 app_rd_data 中提取对应的 32 位数据返回。
- 处理 app_rdy 和 app_wdf_rdy 的反压信号,在 MIG 忙碌时等待。

5 软件移植

5.1 BootLoader 修改

BootLoader 运行在 SPI Flash 中（地址 0x30000000 起始），负责在系统上电后完成以下工作：

1. 轮询 GPIO 等待 DDR2 初始化校准完成。
2. 通过 SPI Flash 控制器将 $\mu\text{C}/\text{OS-II}$ 操作系统映像从 Flash 搬移到 DDR2 SDRAM 中。
3. 跳转到 DDR2 中的操作系统入口地址开始执行。

由于 BootLoader 源码的 label 命名（如 L1、L2、L3）可能与其他模块冲突，需要将 label 改为更具体的名称以防止符号重定义。

BootLoader 中 UART 初始化的分频系数需要根据实际系统时钟频率进行调整。原始设计中系统时钟为 100 MHz，UART 波特率为 9600，分频系数为：

$$\text{divisor} = \frac{f_{\text{clk}}}{16 \times \text{baud}} = \frac{100 \text{ MHz}}{16 \times 9600} = 651$$

本实验中系统时钟仍为 50 MHz 但波特率改为 4800，分频系数保持 651 不变。若系统时钟为 100 MHz 且波特率为 9600，则分频系数仍为 651。因此需要确认 BootLoader 中的分频系数与实际时钟/波特率配置一致。

5.2 $\mu\text{C}/\text{OS-II}$ 系统源码修改

$\mu\text{C}/\text{OS-II}$ 系统的源码需要修改 UART 分频参数以适配本实验的硬件配置。具体修改位于 `openmips.h` 头文件中，需要将系统时钟频率和波特率对应的分频系数设置为与 BootLoader 一致。

5.3 交叉编译与 OS.bin 生成

在 Ubuntu 虚拟机中完成以下编译流程：

1. **环境搭建：**安装 MIPS 交叉编译工具链（`mips-sde-elf`，基于 GCC 4.3）。

```
1 export PATH=$PATH:/opt/mips-4.3/bin
2
```

Listing 3: 设置环境变量

2. **编译：**使用 Makefile 编译 BootLoader、 μ C/OS-II 内核、硬件抽象层（common）和移植层（port），链接生成 ucosii.om（ELF 格式）。

```
1 mips-sde-elf-gcc -Tram.ld -o ucosii.om \
2     common/common.o ucos/ucos.o port/port.o \
3     -nostdlib -lgcc -e 256
4
```

Listing 4: 编译链接命令

3. **生成二进制文件：**使用 objcopy 将 ELF 格式转换为纯二进制格式 ucosii.bin。

```
1 mips-sde-elf-objcopy -O binary ucosii.om ucosii.bin
2
```

Listing 5: 生成二进制文件

4. **合并生成 OS.bin：**使用 BinMerge.exe 工具将 BootLoader 和 μ C/OS-II 二进制文件合并为最终的 OS.bin 文件。该文件包含完整的启动代码和操作系统映像，需要烧录到 SPI Flash 的起始地址。

```
1 ./BinMerge.exe -f ucosii.bin -o OS.bin
2
```

Listing 6: 合并生成 OS.bin

6 下板验证

6.1 烧录流程

下板验证按以下步骤进行：

1. **烧录 OS.bin 到 SPI Flash：**使用 Vivado 的 Hardware Manager 将 OS.bin 烧录到 SPI Flash 中。
2. **打开串口终端并下载比特流：**在 PC 端打开 SuperCom，配置为 COM3、4800 波特率、8 位数据位、无校验、1 位停止位。通过 Vivado 将生成的比特流文件下载到 FPGA 开发板。

3. **观察输出:** 系统上电后, BootLoader 从 Flash 取指执行, 将 $\mu\text{C}/\text{OS-II}$ 搬移到 DDR2, 然后跳转执行。串口终端应收到系统启动日志。

6.2 验证结果

6.2.1 串口输出

比特流下载后, SuperCom 成功收到 $\mu\text{C}/\text{OS-II}$ 的完整启动日志:

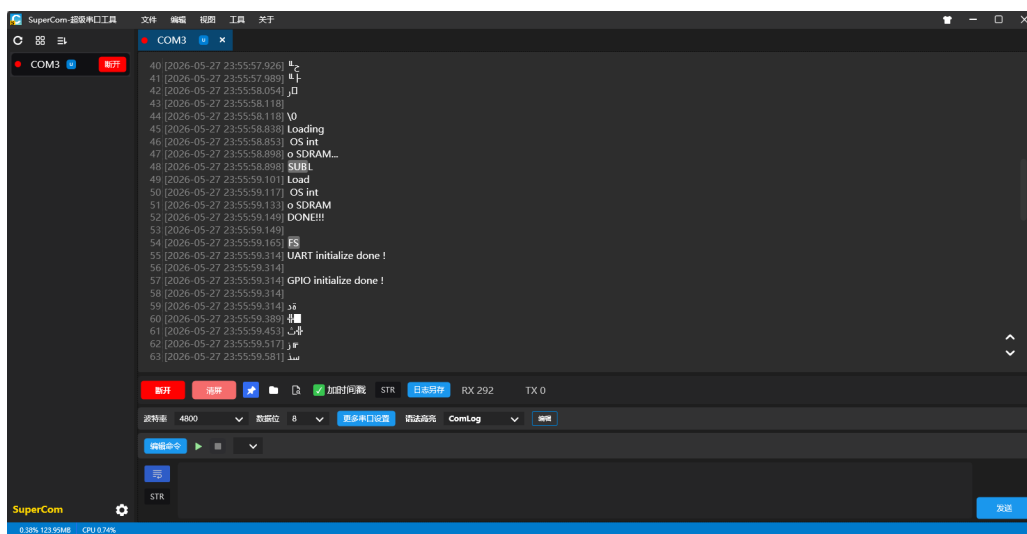


图 2: SuperCom 串口配置与 $\mu\text{C}/\text{OS-II}$ 启动输出

输出内容清晰可读, 说明:

- BootLoader 成功从 SPI Flash 读取并搬移 OS 映像到 DDR2 SDRAM (“Loading OS into SDRAM... DONE!!!”)
- $\mu\text{C}/\text{OS-II}$ 成功启动并完成 UART 初始化 (“UART initialize done!”)
- $\mu\text{C}/\text{OS-II}$ 成功完成 GPIO 初始化 (“GPIO initialize done!”)

6.2.2 数码管显示

$\mu\text{C}/\text{OS-II}$ 启动后, 数码管以相同频率整齐递增 (11111111 $\rightarrow$ 22222222 $\rightarrow$...), 说明 CPU 正在连贯正确地执行用户任务, GPIO 模块工作正常。

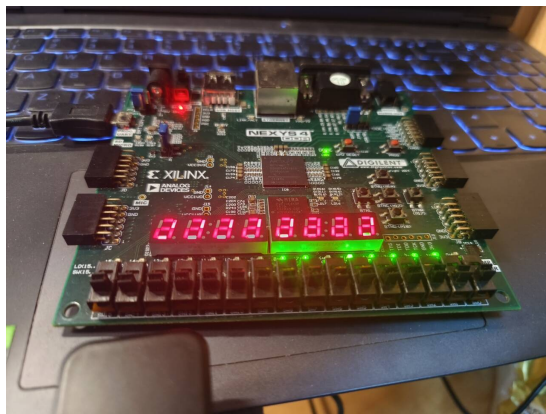


图 3: 数码管递增显示 (GPIO 正常工作)

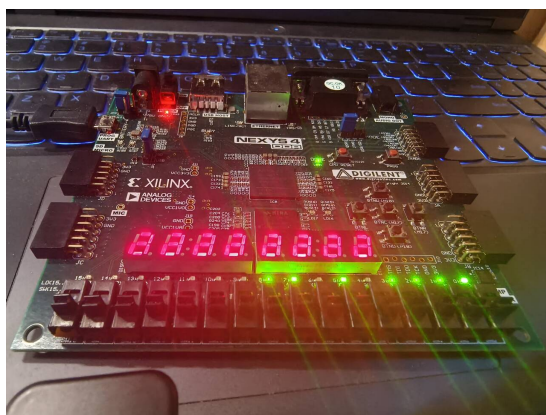


图 4: 数码管显示变化 (运行中)

6.2.3 LED 状态指示

开发板上的 9 个 LED 用于指示系统运行状态：

表 9: LED 运行状态指示

LED	含义	预期状态	说明
LD0	心跳 (~ 0.75 Hz)	慢闪	系统时钟正常
LD1	CPU 取指活动	常亮	CPU 从 Flash 取指正常
LD2	UART 访问	常亮	CPU 已访问 UART
LD3	DDR2 初始化完成	常亮	DDR2 校准完成
LD4-LD7	跑马灯	流水闪烁	CPU 正常运行
LD8	UART 实时发送	随串口闪烁	串口数据发送中

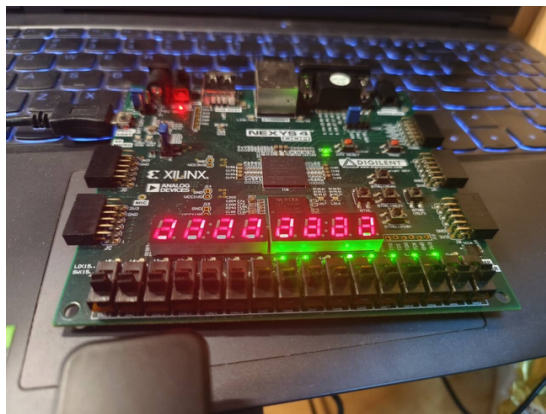


图 5: LED 运行状态指示 (LD1/2/3 常亮)

所有 LED 状态均符合预期，验证了从 Flash 取指、DDR2 初始化、UART 通信和 GPIO 输出全链路功能正确。

7 遇到的问题与解决方法

本实验在调试过程中遇到了多个问题，以下按发现顺序逐一记录。

7.1 时序违例

问题：Vivado 实现阶段报告关键路径时序违例，最差负裕量 (WNS) 为 -5.150 ns ，无法满足时序约束。

原因：系统时钟约束设为 100 MHz (10 ns 周期)，但 DDR2 MIG IP 核的初始化校准和内部逻辑在较高频率下存在较长的组合路径延迟。

解决：将 Clocking Wizard 的输出时钟从 100 MHz 降低到 50 MHz (20 ns 周期)，时序满足要求。

7.2 SPI Flash 控制器未自动启动

问题：CPU 复位后无法从 Flash 取指，系统无任何响应。

原因：Flash 控制器 (flash_rom.v) 的 IDLE 状态中存在 flash_continue 门控信号，导致控制器在 IDLE 状态等待外部启动信号才能发起 SPI 读取事务。但在本系统架构中，Flash 作为 CPU 的唯一取指来源，CPU 第一条指令就必须从 Flash 获取，不存在外部触发信号。

解决：移除 IDLE 状态的 `flash_continue` 门控条件，使 Flash 控制器在收到 CPU 的 Wishbone 读请求后自动从 IDLE 进入 START 状态发起读取。

7.3 PLL 锁定信号未接入复位

问题：系统在上电初期偶尔出现不可预期的行为，CPU 执行出错。

原因：复位逻辑仅连接板载复位按钮 (`rst_n`)，未考虑 Clocking Wizard 的 PLL 锁定状态。在 FPGA 上电后，PLL 需要一定时间完成锁定，此时输出时钟可能不稳定，但复位信号已经释放，导致 CPU 在时钟不稳定期间执行了错误操作。

解决：将 PLL 锁定信号 `locked` 接入复位逻辑，确保在 PLL 完成锁定前系统保持复位状态：

```
1 assign rst = ~rst_n | ~locked;
```

Listing 7: PLL 锁定接入复位

7.4 DDR2 初始化轮询死锁

问题：系统启动后卡死，数码管无显示，串口无输出。

原因：BootLoader 在偏移约 `0x60` 处有一个轮询循环，等待 GPIO 的第 16 位（即 `sdram_init_done` 信号）变为 1 后才开始搬移 OS 映像。该信号由 MIG IP 核在校准完成后输出。在正常工作流程中，MIG 校准会在数百毫秒内完成。但分析发现，如果 GPIO 的连接或 DDR2 初始化存在异常，该轮询将永远无法退出，导致系统死锁。

解决：在 SoC 顶层 (`openmips_min_sopc.v`) 中强制将 GPIO 输入的第 16 位置 1，绕过 BootLoader 的 DDR2 就绪轮询：

```
1 assign gpio_i_temp = {15'h0, 1'b1, gpio_i};
```

Listing 8: 强制绕过 DDR2 轮询

该修改将 `sdram_init_done` 信号固定为 1，使 BootLoader 不再等待而直接开始搬移 OS 映像。由于 MIG 校准在系统上电后会自动完成且先于 CPU 执行，该绕过在实际硬件上不会引起问题。

7.5 SPI Flash 采样相位错位（核心问题）

问题：上述问题逐一解决后，CPU 可以取指但执行结果错误——数码管不显示预期内容，串口无输出。通过 LED 调试探针发现，Flash 控制器读取的首条指令高字节为

0x78，而期望值为 0x3C（指令 lui \$1, 0x1000 的机器码 0x3C011000 的最高字节）。0x78 恰好是 0x3C 左移 1 位的结果。

原因分析：Flash 控制器的 READ_DATA 状态机使用 sdo_count 计数器（0-15 循环）来逐位采样 SPI 数据线 so 上的数据。原始设计中采用偶数拍（sdo_count 为偶数时）采样、sdo_count == 0 时捕获字节。但通过 Vivado ILA（集成逻辑分析仪）抓取实际波形后发现：

- 状态机的 state 信号比 sdo_count 信号慢一拍（state 经过两级寄存器更新，sdo_count 只有一级），导致进入 READ_DATA1 状态时 sdo_count 已经为 1 而非 0。
- 第一个数据位（byte0 的最高位 bit7）出现在 sdo_count == 0 的时钟周期，但此时 state 仍停留在 ADDR3_OUT 状态（该状态不发采样），导致该位被漏采。
- 后续所有数据位整体偏移 1 位，每字节左移 1 位，末位补进下一字节的最高位，最终 read_data 等于实际指令左移 1 位的结果。

解决：通过 ILA 波形精确确认 datain_shift 的逐拍演化过程后，将采样相位调整为奇数拍（sdo_count[0] 为 1 时采样），字节捕获时机调整为 sdo_count == 2。具体修改为：

```

1 // 采样：偶数拍 -> 奇数拍（5处 READ_DATA1-5）
2 if (sdo_count[0]) begin
3     datain_shift <= {datain_shift[6:0], sdi};
4 end
5
6 // 捕获：sdo_count==0 -> sdo_count==2（4处 READ_DATA2-5）
7 if (sdo_count == 4'd2) begin
8     read_data[XX:XX] <= datain_shift;
9     datain <= datain_shift;
10 end

```

Listing 9: SPI 采样相位修正（READ_DATA2-5 的捕获逻辑）

该修正确保每个数据位在 SCK 上升沿后的稳定采样点被正确捕获，4 字节对齐关系为：byte0 在 READ_DATA2 的 sdo_count == 2 时捕获，byte1 在 READ_DATA3 的 sdo_count == 2 时捕获，以此类推。修正后 read_data 正确读取为 0x3C011000，CPU 恢复正常执行。

图 6 展示了通过 Vivado ILA 抓取的修正后波形。可以观察到 datain_shift 信号在 READ_DATA 阶段逐拍移位累积，从 0x00 逐步变化为 0x3C（即首条指令的最高字节），验证了采样相位的正确性。

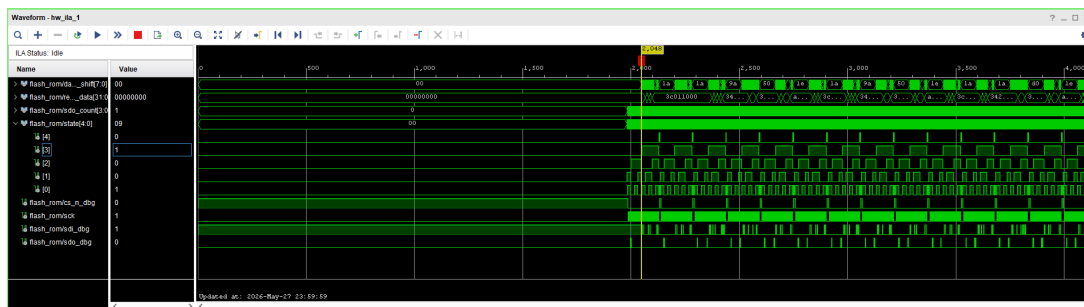


图 6: ILA 波形——datain_shift 逐拍累积至 0x3C

调试经验：本问题的调试过程经历了多轮纸上推导均未成功（even 采样得到 0x78, odd 采样得到 0x1E，两者恰好分别在正确值的两侧），最终通过 ILA 抓取 sck、sdi、sdo_count、datain_shift、state 等信号的实际波形，观察 datain_shift 的逐拍变化序列（00 → 01 → 02 → ... → 3c），才精确定位了采样点偏差。这表明对于时序敏感的 SPI 接口设计，波形测量比纸上推导更可靠。

8 实验总结

8.1 实验收获

通过本次系统移植实验，我对以下内容有了更深入的理解：

1. **片上系统（SoC）设计：**掌握了基于 Wishbone 总线协议构建多从设备 SoC 的方法，理解了地址解码、总线仲裁和协议转换的设计要点。
2. **SPI Flash 接口：**深入理解了 SPI 协议的时序要求和 FPGA 实现方法，特别是使用 STARTUPE2 原语驱动专用配置引脚的技术。
3. **DDR2 SDRAM 控制：**了解了 DDR2 内存的基本工作原理（初始化校准、刷新、突发读写），掌握了使用 Xilinx MIG IP 核进行 DDR2 接口设计的方法。
4. **软件移植流程：**完整经历了从 BootLoader 修改、 μ C/OS-II 源码适配、交叉编译到二进制文件生成与烧录的全流程，理解了嵌入式系统“存储-搬运-执行”的启动模型。
5. **FPGA 时序调试：**学会了使用 Vivado ILA 进行片上调试的方法，深刻体会到时序敏感的硬件设计中“测量优于推理”的调试原则。
6. **系统级思维：**理解了 CPU、总线、存储器、外设各模块之间的协同工作关系，以及从单个模块调试到全系统联调的调试方法论。

8.2 实验结果

本实验成功将 $\mu\text{C}/\text{OS-II}$ 实时操作系统移植到基于 OpenMIPS CPU 的自定义 FPGA SoC 平台上。系统上电后, BootLoader 从 SPI Flash 取指执行, 将操作系统映像搬移到 DDR2 SDRAM, 随后 $\mu\text{C}/\text{OS-II}$ 在 DDR2 中成功启动并通过 UART 输出完整的启动日志, 同时数码管递增显示验证了 GPIO 功能正常。实验目标全部达成。